

Disable Startup Programs

Disable Startup Programs

Raspberry Pi

Temporarily Disable Startup Programs

Docker Basics

Entering the Same Container from Multiple Terminals

Running the script

Permanently Disable the Startup Program

Exiting the Container

Running in the Foreground

Running in the Background

Startup Script

Modifying the Startup Script

Run the Startup Script

Disable the Docker Autostart Script

Restore Docker Startup Script

Since our image is factory-configured to start with a startup program, running other examples without disabling it will result in some abnormal errors!

The method for disabling startup programs on Raspberry Pi and Jetson motherboards using Dofbot is the same!

Raspberry Pi

Temporarily Disable Startup Programs

Docker Basics

This section only introduces some Docker commands; for more detailed information, please refer to the Docker chapter tutorial.

Entering the Same Container from Multiple Terminals

Since our system automatically starts a specific container upon boot, we need to enter the same container to run the script that disables startup programs.

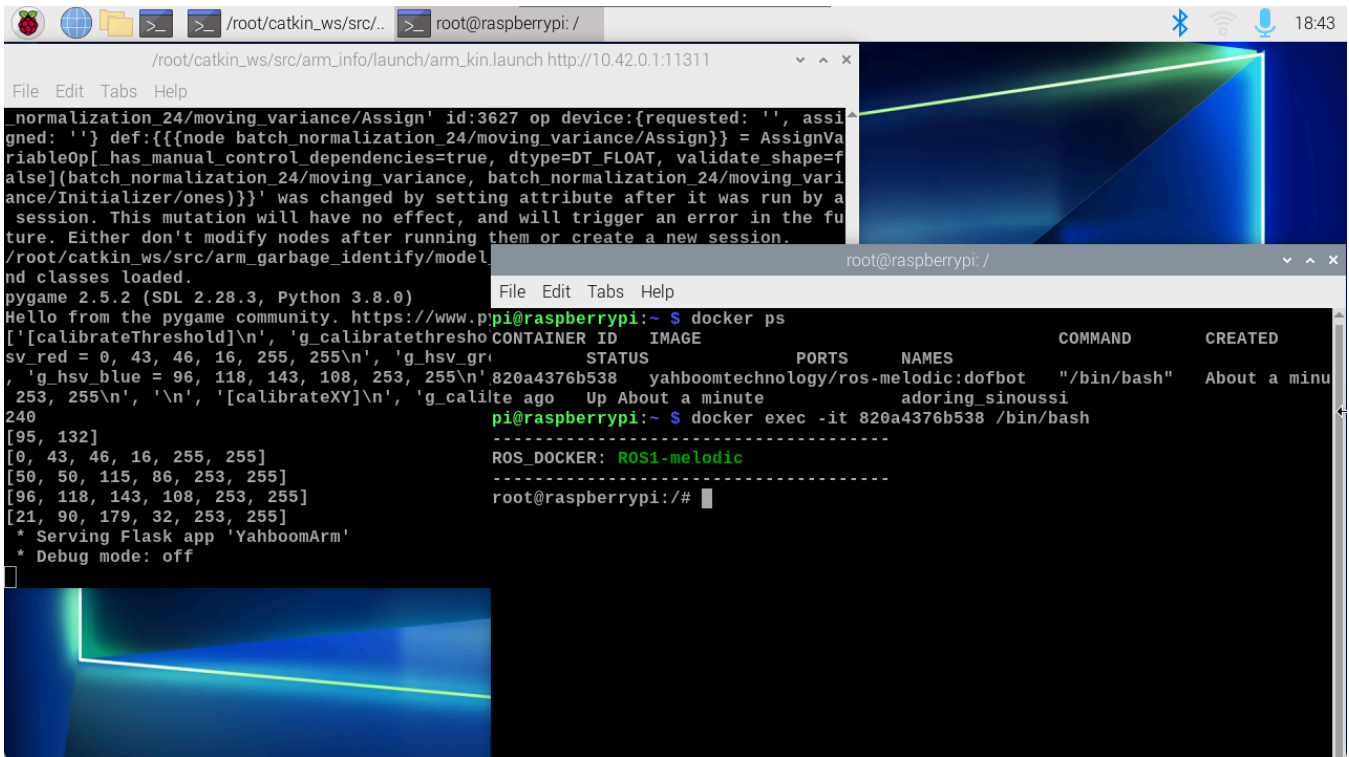
docker ps

Used to list currently running containers.

docker run

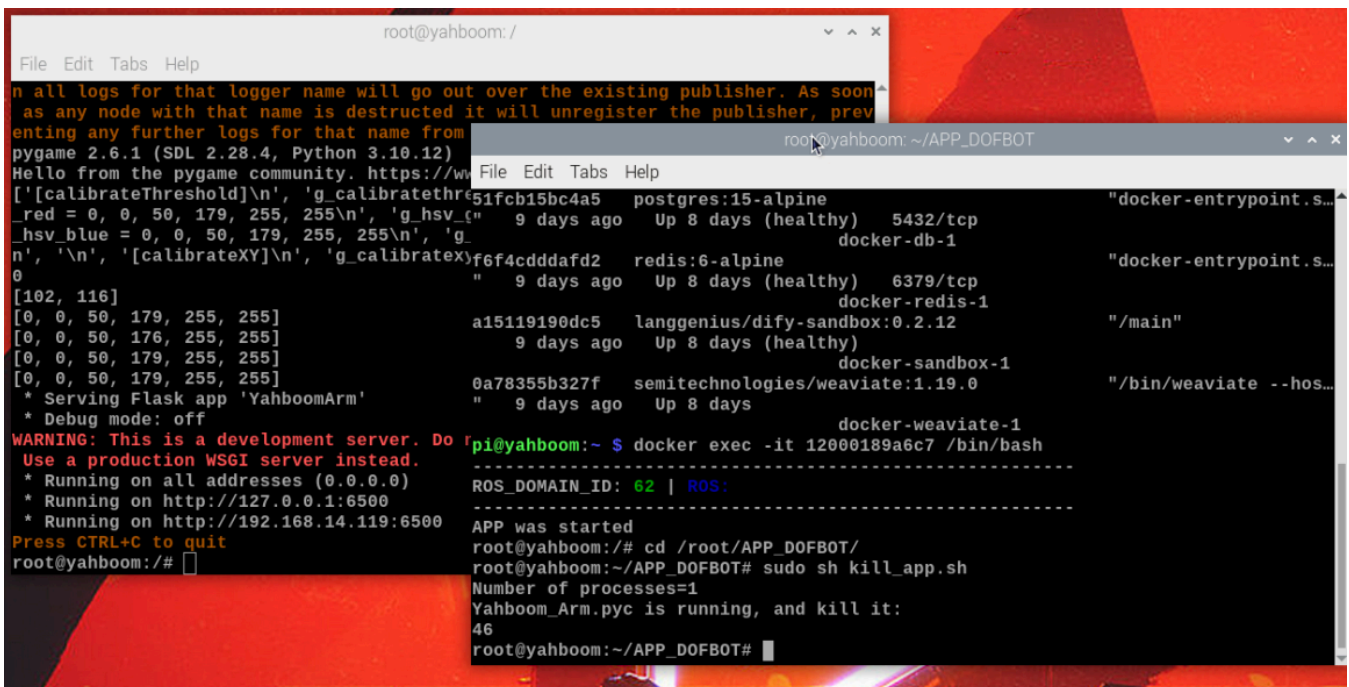
Used to create and start a new container instance.

```
#Start a container in interactive mode and execute the `/bin/bash` command inside the container
docker exec -it 820a4376b538 /bin/bash
```



Running the script

```
cd /root/APP_DOFBOT/
sh kill_app.sh
```

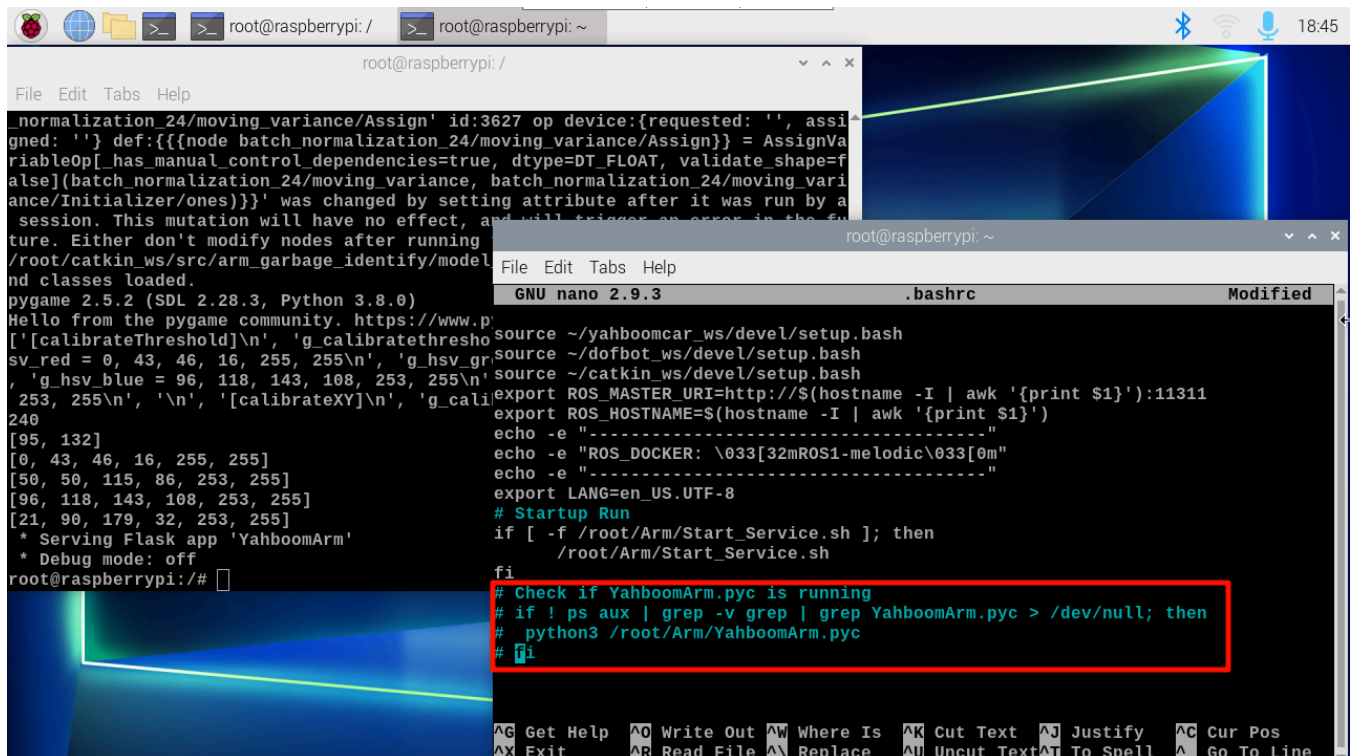


Temporarily stopping a large program only takes effect in the currently running container. If you restart the container yourself using a script, Docker will rerun the large program at startup!

The script that starts the large program is added to the `.bashrc` file inside Docker. Therefore, every time a new container starts, the large program runs at startup. If multiple containers are started and not properly shut down, the large program will display a message indicating that the port is in use!

Permanently Disable the Startup Program

Since our large-scale startup program is configured within the `.bashrc` file, we need to either comment out or delete the lines in `.bashrc` that relate to `YahboomArm.pyc`. This content is located at the very bottom of the file.



```
root@raspberrypi: /
File Edit Tabs Help
_normalization_24/moving_variance/Assign' id:3627 op device:{requested: '', assi
igned: ''} def:{{{node batch_normalization_24/moving_variance/Assign}}} = AssignVa
riableOp[_has_manual_control_dependencies=true, dtype=DT_FLOAT, validate_shape=f
alse](batch_normalization_24/moving_variance, batch_normalization_24/moving_van
iance/Initializer/ones)}}} was changed by setting attribute after it was run by a
session. This mutation will have no effect, and will trigger an error in the fu
ture. Either don't modify nodes after running
/root/catkin_ws/src/arm_garbage_identify/model
nd classes loaded.
pygame 2.5.2 (SDL 2.28.3, Python 3.8.0)
Hello from the pygame community. https://www.py
['calibrateThreshold\n', 'g_calibratethresho
sv_red = 0, 43, 46, 16, 255, 255\n', 'g_hsv_gr
, 'g_hsv_blue = 96, 118, 143, 108, 253, 255\n'
, 253, 255\n', '\n', '[calibrateXY\n', 'g_cal
240
[95, 132]
[0, 43, 46, 16, 255, 255]
[50, 50, 115, 86, 253, 255]
[96, 118, 143, 108, 253, 255]
[21, 90, 179, 32, 253, 255]
* Serving Flask app 'YahboomArm'
* Debug mode: off
root@raspberrypi:/#

GNU nano 2.9.3 .bashrc Modified
source ~/yahboomcar_ws/devel/setup.bash
source ~/dofbot_ws/devel/setup.bash
source ~/catkin_ws/devel/setup.bash
export ROS_MASTER_URI=http://${hostname -I | awk '{print $1}':11311
export ROS_HOSTNAME=${hostname -I | awk '{print $1}'}
echo -e "-----"
echo -e "ROS_DOCKER: \033[32mROS1-melodic\033[0m"
echo -e "-----"
export LANG=en_US.UTF-8
# Startup Run
if [ -f /root/Arm/Start_Service.sh ]; then
    /root/Arm/Start_Service.sh
fi
# Check if YahboomArm.pyc is running
# if ! ps aux | grep -v grep | grep YahboomArm.pyc > /dev/null; then
# python3 /root/Arm/YahboomArm.pyc
# fi
```

Once the modifications are complete, we need to commit a new version of the Docker image:

First, check the currently running containers:

```
docker ps
```

docker commit

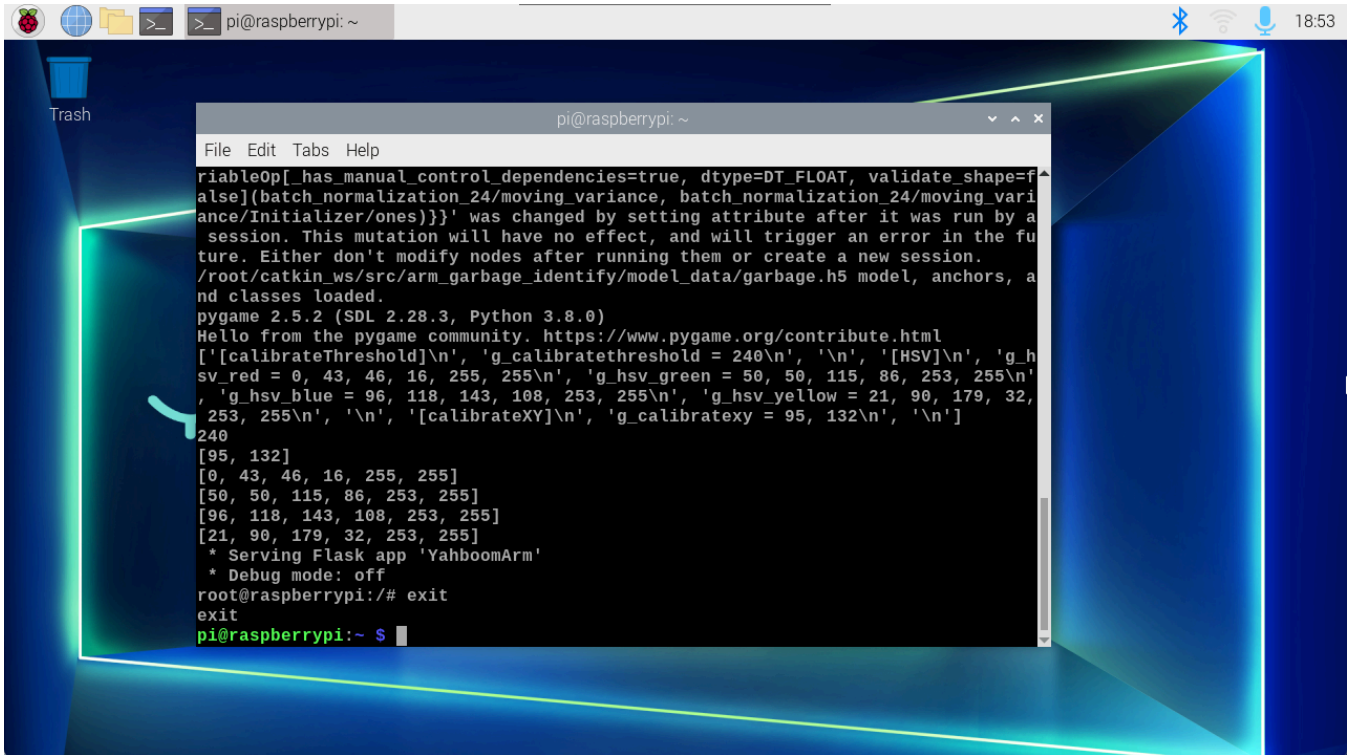
This command is used to save the current state of a container as a new image. Since tags such as `1.0` and `dofbot` already exist within the Docker environment, I will demonstrate this process using the tag `dofbot_v1.0`.

```
# docker commit [OPTIONS] CONTAINER [REPOSITORY[:TAG]]
docker commit 820a4376b538 192.168.2.51:5000/dofbot_ai:v1
```

Exiting the Container

Running in the Foreground

If the container is currently running in the foreground, you can simply type `exit` in the terminal to quit!

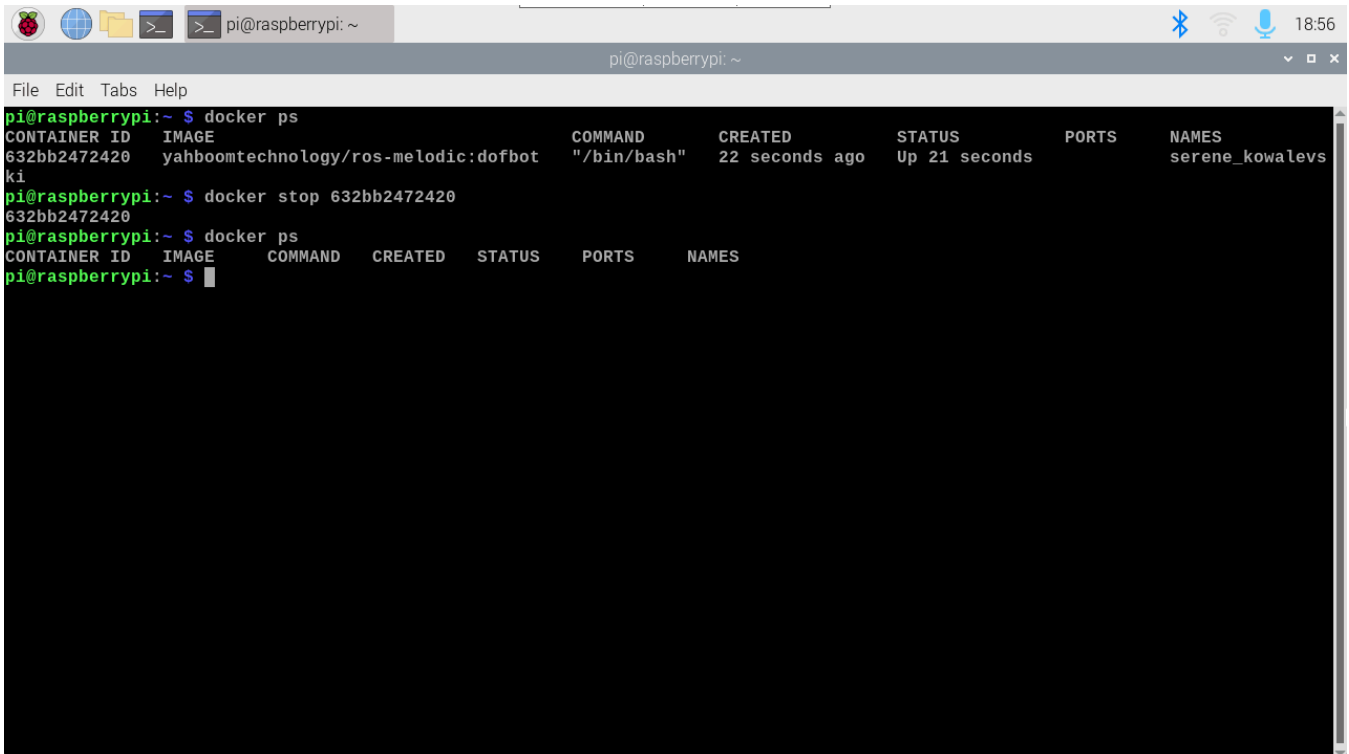


Running in the Background

docker stop

This command is used to stop a specified container.

```
# docker stop [OPTIONS] CONTAINER
docker stop 632bb2472420
```



Startup Script

Modifying the Startup Script

The startup script for automatic execution upon boot: Docker_Ros.sh

```
#!/bin/bash

# wait for the Docker service to start
while true; do
    if systemctl is-active --quiet docker; then
        echo "Docker service has been started"
        break
    else
        echo "The Docker service has not started, waiting..."
        sleep 1
    fi
done

# Docker start
xhost +

docker run -it --rm \
--privileged=true \
--net=host \
--env="DISPLAY" \
--env="QT_X11_NO_MITSHM=1" \
-v /tmp/.X11-unix:/tmp/.X11-unix \
--security-opt apparmor:unconfined \
-v /home/pi/Temp:/root/Temp \
--device=/dev/video0 \
--device=/dev/i2c-1 \
192.168.2.51:5000/dofbot_ai:3.0.1 /bin/bash
```

Update to the new Docker image we just submitted:

```
#!/bin/bash

# wait for the Docker service to start
while true; do
    if systemctl is-active --quiet docker; then
        echo "Docker service has been started"
        break
    else
        echo "The Docker service has not started, waiting..."
        sleep 1
    fi
done

# Docker start
xhost +

docker run -it --rm \
```

```
--privileged=true \  
--net=host \  
--env="DISPLAY" \  
--env="QT_X11_NO_MITSHM=1" \  
-v /tmp/.X11-unix:/tmp/.X11-unix \  
--security-opt apparmor:unconfined \  
-v /home/pi/Temp:/root/Temp \  
--device=/dev/video0 \  
--device=/dev/i2c-1 \  
192.168.2.51:5000/dofbot_ai:v1 /bin/bash
```

Run the Startup Script

```
./Docker_Ros.sh
```

This prevents the large startup application from launching. If you *do* need to launch the large startup application, simply modify the specified container within the `Docker_Ros.sh` script to use the factory default image.

That's all there is to it!

Disable the Docker Autostart Script

Enter the following command to restart:

```
#RaspberryPi5  
sudo rm -rf /home/pi/.config/autostart/Docker_Ros.desktop  
#jetson-nano  
sudo rm -rf /home/jetson/.config/autostart/Docker_Ros.desktop
```



Restore Docker Startup Script

Enter the command below to restart:

```
#RaspberryPi5
sudo cp Docker_Ros.desktop /home/pi/.config/autostart/
#jetson-nano
sudo cp Docker_Ros.desktop /home/jetson/.config/autostart/
```

